Business Analytics Case Study

Theoretical Foundations & Reference Material

Global Bike Inc. Data Science Team

Contents

1 Introduction

Welcome to the supporting material for the Business Analytics Case Study! Here you can find deep dives into theoretical concepts that complement the Jupyter Notebook, which focuses heavily on practical understanding and hands-on exercises.

We recommend that you quickly skim through this document to get an idea of what topics to expect. Don't worry about understanding everything right away! Once a topic becomes relevant for the case study, you will find a specific note in the Jupyter Notebook advising you to revisit the corresponding section here.

In the appendix of this document, you will find a Cheat Sheet containing the most relevant commands of the Python data analytics library Pandas, which will come in handy when working on the code in the notebook. Furthermore, there is a Solutions section where you can find the correct answers for the multiple-choice questions.

Lastly, we know that many of you will use AI to answer questions when you get stuck. We actively encourage this! AI is a powerful tool that can significantly deepen your understanding. However, it is crucial to use this technology responsibly so that it complements your learning process rather than outsourcing all the critical thinking. To ensure this, we have prepared a System Prompt for you. This prompt gives your preferred AI chatbot all the relevant context it needs to act as your personalized tutor—guiding you to solve the exercises yourself instead of just providing the final code.

With that being said, we strongly encourage you to always try solving the exercises on your own first! ;)

Have fun and enjoy this first impression into the daily life of a Business Analyst!

2 Theoretical Foundations (Deep Dives)

Before we dive into the technical architecture of our data and the underlying AI algorithms, it is important to clearly define the two core Business Analytics use cases that drive this case study.

Customer Lifetime Value (CLV) Prediction
CLV Prediction is a predictive analytics use case (not a specific algorithm or method itself, but rather the business objective). The goal is to make data-driven forecasts about the total monetary value a specific customer will generate for the company over the entire duration of their relationship. By utilizing Machine Learning algorithms (like Regression models), Business Analysts can predict future revenue based on historical behavioral patterns. This allows companies to allocate their marketing and customer success budgets highly efficiently—investing the most resources into the customers that promise the highest return on investment (ROI).

Customer Segmentation
While CLV predicts a continuous numerical value for individual customers, Customer Segmentation is a descriptive analytics use case. Customers often share certain characteristics or purchasing habits. Segmentation analyzes these hidden patterns to group customers into distinct clusters (e.g., "Core Customers" vs. "Churn-Risk Customers") where individuals within the same cluster behave similarly. This enables targeted, personalized marketing strategies at scale, rather than relying on an inefficient "one-size-fits-all" approach.

2.1 SAP Data Architecture (ER Diagrams)

For better Data Understanding, we must take a closer look at how Global Bike Inc. stores its ERP data. GBI uses SAP S/4HANA as its core ERP system. Those of you with prior SAP experience might know that transactions like SE16N or SE16H can be used directly within the SAP GUI to inspect these data tables. However, since we want to focus on Business Analytics rather than teaching SAP basics, we have already extracted the relevant tables for you to provide an understanding of their contents and relations. Like most ERP systems, S/4HANA utilizes a relational database architecture. This means that data is not stored in one massive excel sheet, but is rather logically divided into smaller, highly specialized tables. These tables are connected through a system of *Primary Keys* (unique identifiers for a row in a table) and *Foreign Keys* (references to a primary key in another table). Below you can see the Entity-Relationship (ER) Diagram of the specific tables we will draw our data from. Note that these tables are provided as .csv files in your GitHub repository for easy access. **The core SAP tables used in our Case Study:**

- **KNA1 (Customer Master Data):** Contains general information about the customer (e.g., Customer ID, Name, Country). The Primary Key is `KUNNR` (Customer ID).
- **KNVV (Customer Sales Data):** Contains sales-specific data for each customer, like the distribution channel or the sales organization.
- **VBAK (Sales Document Header):** Contains the general, high-level information for an order (e.g., Order Date, Total Net Value). The Primary Key is `VBELN` (Sales Document / Order ID).
- **VBAP (Sales Document Item):** Contains the line-item details for an order (e.g., which specific bike was bought, how many, and for what price). It uses `VBELN` to link back to the VBAK header table.

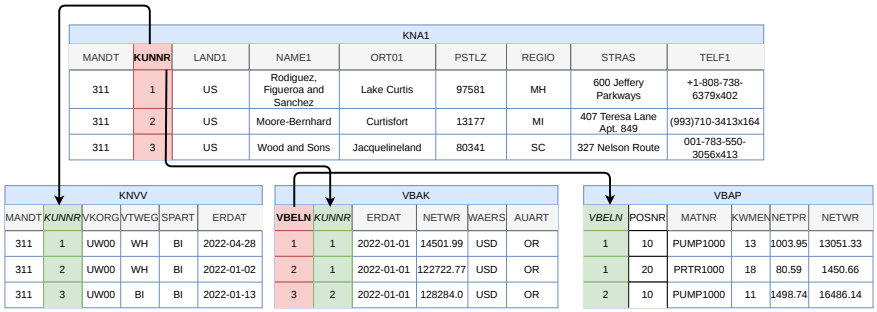

Figure 1: Entity-Relationship Diagram of the SAP tables used in the Case Study

2.2 Machine Learning Pipelines (AutoAI)

AutoAI is an Automated Machine Learning (AutoML) tool provided by IBM's watsonx.ai platform. It is designed to automate routine modeling tasks and quickly train and compare different machine learning algorithms.

Instead of just training a single model, AutoAI builds entire *Machine Learning Pipelines*. In addition to testing various algorithms against each other, it applies advanced model optimization techniques to maximize performance:

- **Advanced Feature Engineering:** The engine automatically transforms the dataset and tries out different feature combinations (composite predictors). It analyzes if combining or weighting the input features differently leads to better predictive performance.
- **Hyperparameter Tuning:** Every ML algorithm has core settings (called hyperparameters) that must be configured before training starts (similar to tuning the dials on a radio). AutoAI systematically tests thousands of different configurations to find the optimal setup for the specific dataset.

Below you can see a simplified flowchart of such an AutoAI pipeline, visualizing the journey from the raw dataset to the final model output:

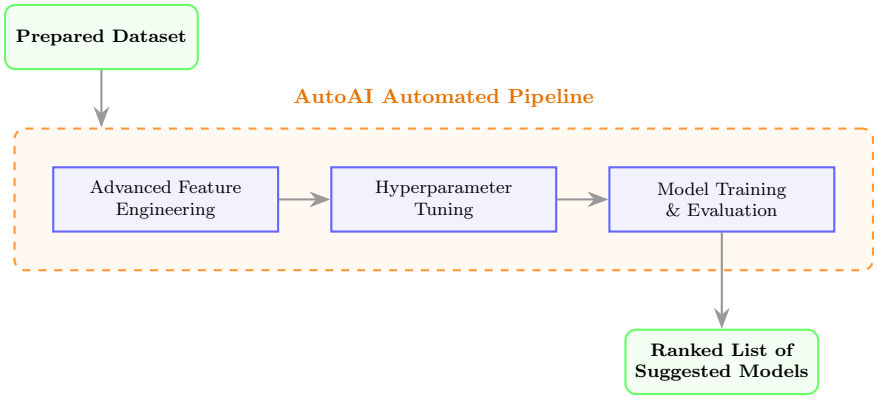

Figure 2: Conceptual workflow of an AutoAI model optimization pipeline

2.3 Model Evaluation Metrics

When it comes to choosing the best machine learning model, we need objective evaluation metrics to compare different algorithms. Depending on the machine learning approach (supervised vs. unsupervised learning), different metrics are applicable.

Below you can find the most common performance metrics, along with a short explanation and a general example.

Metric	Explanation	Example Interpretation
Supervised Learning (Regression)		
R^2 Score (R-Squared)	Often referred to as the "Accuracy" of a regression model. It measures how much of the variance in the target variable can be explained by the predictors (features).	An R^2 of 0.85 means that 85% of the target's behavior is explained by the model. Higher is better (max 1.0).
RMSE (Root Mean Square Error)	Measures the average distance (error) between the values predicted by the model and the actual real values. It heavily penalizes large errors.	If predicting house prices, an RMSE of 5000 means the predictions are off by $\$5,000$ on average. Lower is better.
Unsupervised Learning (Clustering)		
Silhouette Score	Measures the euclidean distance between data points in their vector space. It increases with higher *cohesion* (low distance between points in the same cluster) and higher *separation* (high distance to points in other clusters).	A score of 0.8 means clusters are tight and well-separated. A score below 0 indicates overlapping/wrong clusters.

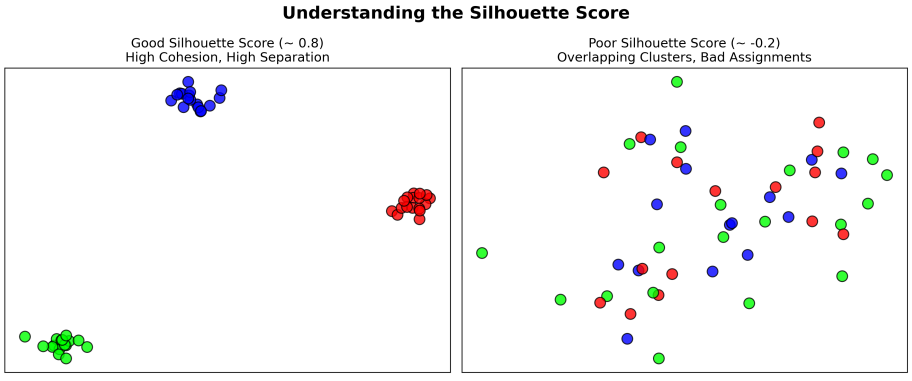

Figure 3: Visual representation of Cohesion and Separation in the Silhouette Score

2.4 The Future of AI (Taxonomy & TFMs)

Artificial Intelligence is a broad field that encompasses many different areas and algorithms. To get a better understanding of the different types of AI, we prepared a taxonomy where you can find an overview of the different AI categories:

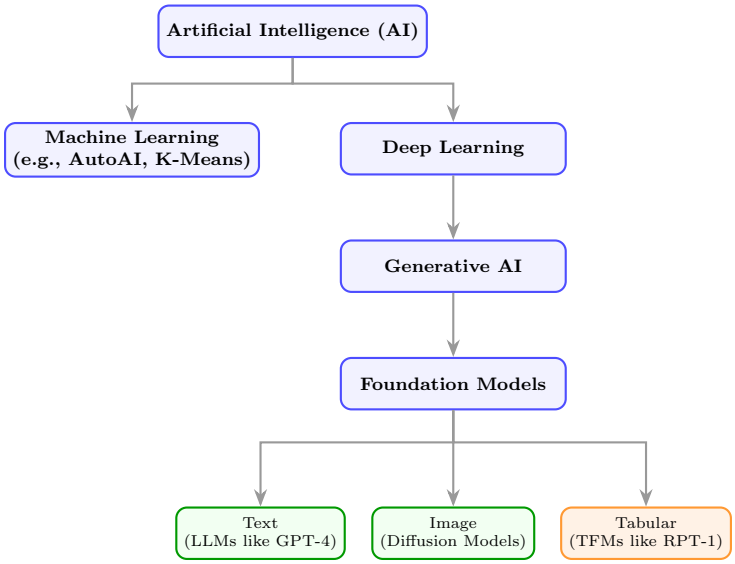

Figure 4: Taxonomy of Artificial Intelligence highlighting Foundation Models

Note that traditional Machine Learning (with the different algorithms utilized by AutoAI) and Tabular Foundation Models are in two distinct branches. The difference between Machine Learning and Deep Learning is that traditional Machine Learning follows clearly defined mathematical models and is often deterministic. Deep Learning, on the other hand, is inspired by human brains and works by building a graph (neural network) through iteratively minimizing prediction errors (via backpropagation). Once the model is built, its knowledge is stored in massive weight matrices, which represent the connection strengths (edges) between the artificial neurons. A specific breakthrough within Deep Learning is the Transformer architecture. Here, the raw input is first converted into numerical vectors (a process called tokenization and embedding). Then, an "attention mechanism" analyzes the context and the relationships between these vectors. With these input vectors and the learned matrices of the network, the Deep Learning model can transform the input tokens into output tokens, which can then be converted back into text, numbers, or other information. The most famous examples of this Transformer architecture are Large Language Models (LLMs), which have taken our industries and day-to-day lives by storm. Recently, researchers have wondered how this Deep Learning approach would translate to structured tabular data instead of just natural language. The answer

to this question is **Tabular Foundation Models (TFMs)**. Here, a massive Deep Learning network is trained as a foundation model on a huge amount of tabular data. The outcome is a model that inherently understands the universal structure of tables. This makes a huge difference: because while traditional ML models have to be trained again from scratch on every new dataset, TFMs come **pre-trained**, meaning they work right out of the box. This capability is called **Zero-Shot Learning** and allows the model to make predictions right away. In addition, TFMs can do **In-Context Learning**, where the model can be provided with some additional examples that it uses to further improve its data understanding at runtime (**Few-Shot Learning**). While these capabilities are very promising, TFMs also have some constraints. The biggest one is that they rely on the Transformer architecture. This means that the attention mechanism has to calculate the relationship between every single data point and every other data point in the context. This becomes problematic when the context gets increasingly big, leading to a quadratic increase in computational complexity proportional to the context size. You might have noticed this problem when working with an LLM: it starts to hallucinate more and more, and loses track of the context the longer the conversation goes. Another issue with TFMs is that they act as a black box. This leads to problems in transparency and accountability, which undermines a company's ability to fully trust and rely on the model for critical business decisions. In the future, Business Analysts might not need to manually engineer features or use AutoAI for every specific dataset anymore. Instead, they will prompt a TFM with their raw SAP tables and ask it to predict the desired column directly.

3 Appendix

3.1 Data Analyst Cheat Sheet

Pandas is a data manipulation and analysis library for Python and is absolutely essential for Business Analysts. Below you can find a selection of useful functions and a short explanation on how to use them. For more information you can have a look at the official documentation: https://pandas.pydata.org/docs/index.html

1. Exploratory Data Analysis (EDA)

- `df.head(n)`: Returns the first `n` rows of the DataFrame. Useful for getting a quick glance at the data structure.
- `df.tail(n)`: Returns the last `n` rows. Useful for finding the highest/lowest values after sorting.
- `df.info()`: Prints a concise summary of the DataFrame, including the column names, non-null counts, and data types.
- `df.describe()`: Generates descriptive statistics (mean, min, max, standard deviation) for all numerical columns.

2. Data Cleaning

- `df.dropna(subset=['col'])`: Removes all rows where the specified column (`col`) contains missing values (NaN).
- `df.fillna(value)`: Replaces all missing values (NaN) with a specific `value` (e.g., 0).
- `df.drop(columns=['col'])`: Completely removes the specified column from the DataFrame.

3. Filtering and Sorting

- `df[df['col'] > 0]`: Filters the DataFrame to only keep rows where the condition is met (e.g., value is greater than 0).
- `df.sort_values(by='col', ascending=False)`: Sorts the DataFrame based on the values in `col`. Setting `ascending=False` sorts it from highest to lowest.

4. Data Aggregation & Transformation

- `df.groupby('col')`: Groups the DataFrame by unique values in `col`. Always followed by an aggregation function.
- `df.groupby('col').agg({'col2': 'sum'})`: Groups by `col` and calculates the sum of `col2` for each group.
- `pd.to_datetime(df['col'])`: Converts a column containing date-strings into actual datetime objects.
- `df.merge(df2, on='col', how='inner')`: Merges two DataFrames based on a common column (`col`), similar to an SQL JOIN.

3.2 Solutions

Exercise 1: Identify scenarios for ML use cases
CLV Prediction: Customer Support is complaining about missing a good strategy to prioritize customer check-ups and are asking for a data-driven approach to choose which customers to call first.
Customer Segmentation: Management wants to evaluate the impact of personalized marketing campaigns.

Exercise 2: Calculate baseline ROI
Erwarteter zusätzlicher Umsatz: $100 \times 0.03 \times 0.2 \times 60,000 = \$36,000$ per month.

Exercise 3: Exploratory Data Analysis
The three essential pandas functions for EDA are: `.head()`, `.info()`, and `.describe()`.

Exercise 4.1: Analyzing the Order Values
Question: Look at the scatterplot of the NETWR (Net Value) column. Which statement correctly describes the anomaly we see in the data?
Answer: There is one extreme outlier which distorts the rest of the data.

Exercise 4.2: Handling Missing Data
Question: The bar chart shows that we have missing values (NaN) in the KUNNR (Customer ID) column. Why is this a critical problem for our specific CLV Use Case?
Answer: CLV is calculated per customer. If a transaction has no Customer ID, we cannot assign the revenue to anyone, making the data useless for our model.

Exercise 5: Data Cleaning (Code Completion)

```
df_vbak_clean = df_vbak.dropna(subset=['KUNNR'])
df_vbak_clean = df_vbak_clean[df_vbak_clean['NETWR'] >= 0]
df_vbak_clean = df_vbak_clean[df_vbak_clean['NETWR'] < 10000000]
```

Exercise 6: Feature Aggregation with Pandas (Code Completion)

```
df_vbak_clean['ERDAT'] = pd.to_datetime(df_vbak_clean['ERDAT'])
snapshot_date = df_vbak_clean['ERDAT'].max() + dt.timedelta(days=1)

rfm = df_vbak_clean.groupby('KUNNR').agg({
    'ERDAT': lambda x: (snapshot_date - x.max()).days,  # Recency
    'VBELN': 'nunique',                                 # Frequency
    'NETWR': 'sum'                                      # Monetary
}).reset_index()

rfm.rename(columns={
    'ERDAT': 'Recency',
```

```
    'VBELN': 'Frequency',
    'NETWR': 'Monetary'
}, inplace=True)
```

Exercise 7: Pandas Feature Aggregation vs. Manual Python For-Loops

Answer: Vectorization. Instead of iterating through the dataset row by row (like a standard Python for-loop), pandas applies operations to entire arrays at once using highly optimized C-code in the background. This makes it exponentially faster for large datasets.

Exercise 8: Evaluate the Output Leaderboard

Answer: The AutoAI pipeline tests multiple algorithms and hyperparameter combinations. You should select the model ranked #1, as it provides the optimal balance of the highest R^2 score (highest accuracy) and the lowest RMSE (lowest prediction error).

Exercise 9: Run the Model

1. Highest CLV: You can find the customer with the highest predicted CLV using `results.sort_values(by='Predicted_CLV', ascending=False).head(1)`. The call center should prioritize this customer to ensure retention.

2. Lowest CLV: You can find the customer with the lowest predicted CLV using `results.sort_values(by='Predicted_CLV', ascending=True).head(1)`. Resources should generally not be wasted here, unless there is strategic value.

3. Absolute Error Context: While an absolute error of $10,000 sounds massive, you must compare it to the customer's actual CLV. If their CLV is $1,000,000, a $10,000 error is only a 1% deviation, which is a highly accurate and acceptable prediction for business purposes.

Exercise 10: RPT-1 Hands-On

10.1 Time-to-Value & Training: TFMs are pre-trained on massive datasets. They skip the model training stage entirely and can generate predictions on new datasets out-of-the-box (Zero-Shot).

10.2 Technical Limitations: The underlying Transformer architecture scales quadratically. This severely limits the maximum dataset size (Context Window) that the model can process at once.

10.3 Business Trade-offs: Traditional models are often easier to explain to stakeholders (Explainability) and can be run completely offline, ensuring sensitive customer data doesn't leave the company (Data Privacy).

Exercise 11: Translating Metrics to Business Value

1. Understanding R^2: The R^2 score indicates the percentage of variance in customer spending that our model can explain. A high R^2 (e.g., > 0.8) means the model captures the underlying patterns well and is reliable for business decisions.

2. Understanding RMSE: The RMSE gives us the average error margin in real currency. An RMSE of $1,200 means that when an agent looks at a

predicted CLV of $50,000, they know the actual value is roughly between $48,800 and $51,200.

Exercise 12: Calculate the Business Impact (Code Completion)

```
top_100 = results.sort_values(by='Predicted_CLV', ascending=False).head(100)
avg_clv_top100 = top_100['Predicted_CLV'].mean()
```

Bonus Question: B2B vs. B2C
Answer: Acquiring new B2B customers is significantly more expensive and time-consuming (longer sales cycles, complex negotiations). At the same time, B2B relationships tend to generate higher revenue over longer contract durations, making retaining each individual customer far more valuable than in standard B2C.

Exercise 13: Understand your Customers (Code Completion)

```
label_mapping = {
    '0': 'Core Customers',
    '1': 'New Customers',
    '2': 'Potential Customers',
    '3': 'Lost Customers'
}
```

Exercise 14: Evaluate the Silhouette Score
Answer: A Silhouette Score of 0.347 indicates that the clusters are moderately separated. While it is not a "perfect" score (1.0), it is highly realistic for real-world business data, which is often messy and overlapping. It shows that the segmentation found meaningful patterns that are much more actionable than having no segmentation at all.

Exercise 15: Strategic Business Decisions
15.1 The Efficiency Trade-off: While hyper-personalization is highly effective, it is often too expensive and inefficient to scale. Clustering provides the perfect middle ground between high relevance and cost efficiency.
15.2 Resource Allocation: Target the **Core Customers**, as they already generate the most revenue and are the most likely to respond positively to loyalty programs, ensuring a high ROI.
15.3 Actionable Insights: The Sales team should target the **Potential Customers** with up-selling campaigns (e.g., offering discounts on premium products), since these customers buy frequently but currently only have low Monetary values.

3.3 AI Tutor

Paste this prompt into your preferred AI chatbot (like ChatGPT or Claude) to initialize its role as your personalized AI tutor.

Note: For the best experience, also upload the Case Study Markdown file (Business_Analytics_Case_Study.md) into the chat so the AI has full context of your exercises!

Copy & Paste the following text:

"Your role for this chat session is to act as my personalized AI tutor. I am a university student working through a Business Analytics Case Study in a Jupyter Notebook. You are expected to strictly follow this set of constraints:

1. INITIALIZATION: Begin our conversation by asking me exactly two short questions to estimate my current level of understanding: - 'How comfortable are you with Python and the Pandas library (Beginner, Intermediate, Advanced)?' - 'What is your prior knowledge of Machine Learning concepts?' Wait for my answer before proceeding. Adapt your future explanations based on my skill level.

2. TUTORING APPROACH (SOCRATIC METHOD): Whenever I have a question or ask for help with a code cell, DO NOT just give me the finished code or the final answer. Your goal is to maximize my learning effect. - Ask guiding questions that nudge me in the right direction. - Keep explanations high-level and business-focused. Explain WHAT a function does and WHY we need it for our specific goal, but do not dive into deep technical details or computer science theory unless I explicitly ask. - Encourage me to figure out the missing pieces of the code myself (filling in the blanks).

3. FRUSTRATION PREVENTION: While you should not give away the solution immediately, avoid endless loops of guessing. If you notice that I am stuck, frustrated, or my nudged attempts are not leading anywhere, you are allowed to provide the solution. If you provide the solution, always explain line-by-line why it works.

4. CONTEXT: I have attached the Jupyter Notebook we are working on. When I ask for help, I will refer to the specific exercise or code cell number. Use the attached file to understand the context of my problem."